

Graveyard Shift

Graveyard Shift (*student proposal*)

Assessment: ♦ Fit with adjustments · **Category:** Games · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

Proposal (as submitted): *Graveyard Shift* is a darkly-humorous, fast-paced strategy game: you run a supernatural restaurant in a graveyard serving ghosts, zombies and ghouls. The challenge is **seating customers by kind** (ghosts with ghosts, etc.) — mix them and tensions rise into **chaotic riots**, which leave **remains** you must collect as **ingredients** for the next day's menu. The loop is service → chaos → resource-gathering: mis-seated customers agitate faster (earlier riots); fail to gather enough ingredients and the restaurant risks financial collapse. Named technical features: an **event-based sprite animation system** (animation events call functions at specific frames — e.g. a throw frame spawns a projectile + plays a SFX, keeping animation/gameplay/audio in sync without hard-coded timings); a reusable **FSM template** (start/update/exit) for scenes, player and customers; an **entity-component system** to avoid inheritance hierarchies; and **object pooling** for the many projectiles (fixed pool, reused on out-of-bounds/hit). Loop: a real-time C++ fixed-timestep loop using delta-time — each update advances time-of-day, updates each customer's mood, spreads agitation to nearby customers, moves projectiles, checks collisions and updates ingredients; a mixed table drops moods faster, and once mood passes a limit the customer enters the riot state and throws projectiles. Built for Android, C++ rendering via OpenGL ES connected to a Kotlin app through JNI. Tools: an **animation tool** (load sprite sheets + animation data — triggers, frame rate, start/end frame — via JSON so the team adjusts animations without code) and an **entity data editor** (create customer types, dishes and entities by choosing components and setting values like mood/patience/speed, so UXGD teammates make e.g. an impatient ghoul without recompiling). Design features: time-of-day mood/riot risk; ingredient collection as the player's "health"; seating arrangement (same-kind vs mixed); and a mood + personality system (personality affects how fast mood drops). **Monetization:** a list of in-app purchases — items to collect ingredients (vacuum), **extra patience** / **riot-delay** (TV comedy, calming dish), **larger ingredient-**

collection area, **temporary invincibility**, and **continue/revive** at the current level.

Why "with adjustments" (condition for week-5 approval): the **monetization model is pay-to-win** and breaks the module's clean-monetization rule. Buying **more patience / fewer riots, invincibility, larger collection radius** and **revives/continues** all sell gameplay advantage and reduce difficulty for money — exactly what the rule forbids (no pay-to-win, no manipulative loops). **Rework it to cosmetics-only / non-advantage:** restaurant and customer **skins/decor**, cosmetic collector tools, and (optionally) a one-time premium unlock — remove or re-base the patience/invincibility/radius/revive items so they don't confer a competitive or difficulty advantage. This is owned by the M2 Design goal D8. Everything else here is strong: the ECS + FSM + event-based animation + object-pooled projectiles + fixed-timestep mood/agitation simulation is a genuinely solid engine plan, and the two content tools (animation tool, entity data editor) fit the module's tools requirement well.

References:

- [Diner Dash: Hometown Hero — Wikipedia](#) (the seating-and-mood time-management restaurant game: seat customers by matching type, each group has a patience/mood meter that drains, and mis-management costs money — the seat-by-kind + mood precedent)
- [Overcooked on Steam](#) (a chaotic, fast-paced restaurant game where impatient customers "storm out in a huff" if service falters — the chaos-under-time-pressure restaurant precedent)
- [Cook, Serve, Delicious! on Steam](#) (a restaurant sim built on planning a menu, gathering/serving dishes and managing customers under pressure — the serve-and-manage-economy precedent)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	Graveyard Shift
External Database	✓
Authentication »	✓
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✕
Networking System	●

How each requirement applies:

Platform fit: **Main:** Native mobile (Native Android — C++ via JNI rendering through OpenGL ES, Kotlin app shell; device: Pixel 9a) — the Native PC build is the development/tools vehicle (the animation tool + entity data editor).

Extended: Firebase + a Linux server the team sets up by M2 (full use T2) for accounts, high scores and the (reworked) store.

Front end: A 2D top-down restaurant renderer — the graveyard restaurant, seated customers (ghosts/zombies/ghouls with personality/mood tells), flying

projectiles, remains/ingredients, and the day-time / mood / money HUD — driven by the event-based animation system. **Languages:** Native — Kotlin Native Android UI + the C++ engine via JNI (Android Studio); OpenGL ES for the realtime 2D in a GLSurfaceView; Web/desktop — SDL2/3 + OpenGL ES for the PC dev/tools build.

Back end: The fixed-timestep C++ loop (advance time-of-day, update each customer's mood, spread agitation to tablemates, move projectiles, check collisions, update ingredients), the FSM (scenes + player + customer states), the ECS entities, and the object-pooled projectile system; plus accounts/scores/store. **Languages:** C++ (engine); Firebase-backed services on the Extended tier.

Backend scope: Local now, server-ready (T2) — the accounts / scores / store seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the C++ loop is load-bearing and well-specified — per fixed-timestep update it advances the day, updates and **spreads mood/agitation** across mixed tables, moves projectiles, resolves collisions and updates ingredients, transitioning customers into the riot state past a mood threshold.
- *Data-driven:* the ECS + the entity data editor make customer types, personalities, dishes, mood/patience/speed values and animation data all JSON — a new customer or dish is a data change (a UXGD teammate authors it).
- *2D:* top-down sprite rendering of the restaurant, customers, projectiles and remains + the event-based animation system + HUD (custom-rendered). *3D:* not needed. *Networking:* single-player; accounts/scores/store are async client↔cloud (Application Communication ✓), so a Networking System is optional (●).
- *Custom tool:* the **animation tool** (sprite sheets + triggers/frame-rate/start-end via JSON) **and** the **entity data editor** (compose entities from components + values) — two genuine content-authoring tools.
- *External DB:* accounts, high scores and the (reworked) store. *Auth »:* accounts. *Analytics »:* optional — riot/failure/economy telemetry for balancing.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic** — the fixed-timestep loop + the reusable FSM (scenes/player/customers).

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a fixed-timestep (delta-time) loop advancing time-of-day, mood/agitation, projectiles, collisions and ingredients, with a reusable start/update/exit FSM template; C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the customer AI (M1G8) and event animation (M1G9) run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System** — the ECS (compose entities from components).

Tech & dependencies

- **Tech:** An entity-component system in C++ (transform, collision, physics, mood, personality...) with customer types, dishes, personalities and their values in JSON, hot-reloadable — a new customer/dish is a data change; the entity data editor authors this.
- **Prerequisites:** G1 (loop & system update order).
- **Feeds into:** M2G2 (the entity data editor), M1G8 (customer data), M2G13 (economy).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui overlays (mood/agitation, FSM state, pool usage), Tracy profiling of a full restaurant of customers + a projectile storm, Catch2

tests for the mood/agitation-spread and collision, GitHub Actions Windows+Android matrix.

- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening); the object-pooling budget M2G1 validates.

- **G4 2D Graphics** — the top-down restaurant/customer/projectile renderer + HUD.

Tech & dependencies

- **Tech: Web build:** Sprite/tile renderer (restaurant, seated customers, projectiles, remains), a camera, batching, and the day/mood/money HUD; SDL2/3/OpenGL ES, GLSL, stb_image, FreeType. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G1 (loop & window), G9 (animation drives the sprites).
- **Feeds into:** M2G4 (riot/day-night VFX polish); every visible frame.

- **G8 Enemy AI & Pathfinding** — the **customer mood / patience / agitation-spread / riot FSM** (the gameplay heart).

Tech & dependencies

- **Tech:** Per-customer FSM (waiting → agitated → riot) with mood driven by seating (same-kind vs mixed), personality (patience) and time-of-day, agitation spreading to tablemates, and riot behaviour (throw projectiles); tuned from data.
- **Prerequisites:** G1 (tick), G2 (customer/personality data).
- **Feeds into:** M2G13 (economy/day-cycle systems); the whole seat-or-riot loop.

- **G9 Animation System** — the **event-based animation** system (animation events call functions at specific frames — spawn projectile + SFX on the throw frame).

Tech & dependencies

- **Tech: Web build:** A sprite-animation system driven by JSON animation data (triggers, frame rate, start/end frame) where named frames fire callbacks (spawn projectile, play SFX), keeping animation/gameplay/audio in sync; SDL2/3/OpenGL ES. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.

- **Prerequisites:** G1 (loop), G2 (entities to animate).
- **Feeds into:** M2G2/M2G6 (the animation tool edits this data); projectile spawns; the game's juice.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation — object pooling** for projectiles + on-device validation.

Tech & dependencies

- **Tech: Web build:** Object-pooled projectiles (pre-allocated, reused on out-of-bounds/hit), Tracy profiling of a projectile storm, ASan on MSVC, and on-device validation (Pixel 9a); the Firebase/Linux-server seam (accounts + scores + store) set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The Native-Android deliverable + a stable frame budget during a full riot.

- **G2 ★ Content Editor — Core (Entity Data Editor)** — compose customer types, dishes and entities from components + values (mood/patience/speed).

Tech & dependencies

- **Tech:** Dear ImGui + the shared renderer: pick components and set values to create customer types/dishes/entities, with round-trip JSON serialization and a preview, usable by a UXGD teammate without recompiling.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's SDL2/3/OpenGL ES renderer (preview).
- **Prerequisites:** G2 (the ECS/data model), M1G4 (rendering for preview).
- **Feeds into:** All customer/dish content; M2G6 (the animation tool); the custom-tools showcase.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android Debug builds, tests (mood/agitation,

collision, data loading) and packaging via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.

- **Prerequisites:** G3 (CI & test foundation).
- **Feeds into:** Release/showcase builds of every goal.

- **G6 Content Editor — Advanced Tool Features (Animation Tool)** — the **animation tool** on the event-based system (load sprite sheets + animation data/triggers via JSON).

Tech & dependencies

- **Tech:** Dear ImGui tool to load sprite sheets, set frame rate / start / end frames and place animation triggers (frame events → spawn/SFX), serialized to JSON, with live playback preview.
- **Prerequisites:** M1G9 (the event-based animation system), M2G2 (editor patterns).
- **Feeds into:** Fast animation authoring without code; the content showcase.

- **G13 Gameplay Systems II** — the **seating / mood / ingredient economy + day-cycle + financial-collapse** strategy systems.

Tech & dependencies

- **Tech:** C++ systems over the ECS: the seating-arrangement rules, the ingredient-from-remains economy (ingredients as the player's "health"), the money/financial-collapse loss condition, and the time-of-day riot-risk curve — all balancing-data-driven.
- **Prerequisites:** M1G8 (mood/riot), M1G2 (entities/economy data).
- **Feeds into:** The full service→chaos→gather loop at showcase; T2 content depth.

- **G8 Firebase — Accounts, Leaderboards & Analytics** — accounts, high scores, the (reworked) store and analytics.

Tech & dependencies

- **Tech:** Firebase Auth accounts, Firestore for high scores + the cosmetic store (post-rework), and optional Analytics, via the REST API from C++.
- **Prerequisites:** G2 (score/store data model).
- **Feeds into:** Competitive high scores; the reworked (cosmetic) store; balancing telemetry.

Track Goals & Team Composition

Recommended composition: 6 CS + 2 Design + 1 Art (9) — this is a **systems/strategy** game, so the design surface is large (the seating/mood/agitation/riot rules, the ingredient economy, the day-cycle riot-risk curve, personality tuning, and the reworked monetization), fitting two designers (the proposal already names "UXGD teammates"); one artist can carry a stylised spooky-comedy roster (the ghost/zombie/ghoul factions + personality tells, the restaurant, dishes, projectiles/remains and riot VFX) (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). If the customer/faction art volume outgrows one artist, trade the 2nd designer for a 2nd artist. *Scope note (advisory):* the ECS + FSM + event-animation + object pooling + mood/agitation sim + economy + two tools is a full but well-structured scope — the seat→mood→riot→collect loop with the 3 factions, the two tools and the economy is the M1/M2 floor; broad personality/dish variety and the day-cycle depth are the stretch.

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the seat → mood → riot → collect loop, the darkly-comic tone and the financial-collapse stakes; primary benchmark e.g. Diner Dash / Overcooked, with a Singapore benchmark for the local-awareness criterion)
- M1D2 Core Gameplay Loop Prototype (a greybox proving seat-by-kind → agitation → riot → collect-remains is fun) · prototype in Unity
- M1D3 User Research, Personas & Competitive Analysis (teardown of Diner Dash / Overcooked / Cook Serve Delicious; findings into the FDD)
- M1D5 Level & Progression Design (the day-cycle difficulty ramp and the stage progression)
- M1D6 Systems & Economy Design (the mood/agitation-spread, the ingredient-from-remains economy, money and the personality/patience model — every system with a purpose)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness

- M2D3 Systems Balance & Tuning — Data-Driven (mood/agitation thresholds, riot pacing, ingredient/money balance via the team's JSON)
- M2D5 Playtesting & Iteration Program (test riot readability and the difficulty curve, triage, re-test)
- M2D8 Monetization & Scope Integrity Design — the **required rework**: remove pay-to-win (patience/invincibility/radius/revive), restrict real-money to cosmetics; clean-monetization + a scope-integrity statement
- **UI/UX M1** (graded by the Design instructor for the Games category):
 - M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the seating view, the mood/agitation tells, the day-time clock and the ingredient/money HUD.
 - M1U2 UX Flows & Wireframes — Menus → HUD — the seat → serve → riot → collect → next-day flow wireframed end to end.
 - M1U3 Accessibility & Readability Foundations — legible mood/agitation cues, contrast and touch targets on a phone.
- **UI/UX M2:**
 - M2U1 ★ Functional UI & Usability — seating, service, collection and shop screens navigable unaided on the Native-Android target.
 - M2U2 ★ UX Validation & Final Polished UI — validated, polished seating/riot/collection UX tested with players on the Pixel 9a.
 - M2U3 Menu & HUD Motion Design — mood/agitation, riot-start and day-change transitions animated so state reads clearly.
 - M2U4 Onboarding & Tutorial Design — teach seat-by-kind → riot → collect in the first day without a wall of text.
- **Art M1:** 1 artist (stylised spooky-comedy roster + restaurant + VFX)
 - M1A1 ★ Art Bible / Proof of Concept (the darkly-comic visual direction — palette, the ghost/zombie/ghoul faction language, readability of mood/riot)
 - M1A2 Character Concepts & Model Sheets (the 3 factions + personality tells + the player, with mood/riot expression sheets)
 - M1A3 Environment, Props & VFX Concepts (the graveyard restaurant, dishes, projectiles, remains/ingredients + riot VFX concepts)

- **Art M2:**

- M2A1 ★ Critical Asset Production & In-Engine Visual Quality (customers, restaurant, dishes, projectiles + HUD implemented and consistent)
- M2A2 Character Animation — Extensive (idle/agitated/riot/throw cycles per faction, driven by the event-animation system)
- M2A5 VFX — Critical Effects & Transitions (riot chaos, projectile impacts, remains/ingredient drops, day/night transitions)

- **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (spooky-comic BGM, seat, agitate, riot/throw, collect and day-change cues), suitable and correctly triggered (the throw-frame SFX fires from the animation event).
- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback for a projectile-storm mix.

- **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — standard Games floor (spooky-comic BGM + seat / agitate / riot / throw-impact / collect / day-change SFX, ≥6 types), scaling cleanly with riot intensity.
- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing; frame-event-triggered cues from the animation system).
- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the animation tool + entity data editor), prototype demo, team post-mortem (wk14)